

Diapositiva 1 y 2 - Presentación TFG

Diapositiva 3 - Introducción -Hua

The problem we found is the imbalance between retail investors and big institutions when it comes to market information and tools. Institutions have far more funds, and with them more data, more analysts and better methods to analyse the market than a retail investor has. Our goal is to bridge that gap: to let an investor without a technical background make informed decisions, without needing machine-learning skills.

Diapositiva 4 - Objetivos - Eric

Como ha dicho mi compañero, el objetivo principal de la aplicación es permitir a los usuarios usar modelos de machine learning para la predicción del mercado financiero sin la necesidad de tener conocimientos técnicos previos sobre machine learning,

Para ello, queríamos ofrecer un sistema de creación de modelos sin código, de modo que el usuario pudiera crear, configurar y utilizar modelos directamente desde la aplicación.

También se buscaba que la gestión de los modelos fuera sencilla, permitiendo organizar y agrupar aquellos modelos que estuvieran relacionados en proyectos.

Otro de nuestros objetivos era proporcionar capacidad de predicción real. Para ello, la aplicación ofrece diferentes características y configuraciones que permiten crear modelos capaces de realizar predicciones sobre el mercado.

Por último, queríamos que las predicciones y los informes generados fueran completos pero al mismo tiempo fáciles de entender e interpretar, de forma que el usuario pudiera utilizar los resultados en su toma de decisiones.

Diapositiva 5 y 6 - Funcionalidades - Hua

Moving on to what the application does. The main feature is creating the models: the user builds a custom market-prediction model through a wizard we built, which includes the feature selection we'll explain later on. Once the model is created we train it, and a trained model predicts on the latest market data. Models can be grouped into projects, and a project produces a single report that reads related models together.

Diapositiva 7 y 8 - Arquitectura - Javier:

Respecto a la arquitectura de nuestra aplicación utilizamos una arquitectura de servicios. Hemos implementado manualmente diferentes servicios aislados entre sí. Para tener una arquitectura limpia y escalable hemos decidido aislar los servicios según sus responsabilidades. Comenzamos con la aplicación web, este servidor contiene todo el frontend y toda la lógica de autenticación y negocio de la aplicación. Luego está el servicio de los modelos que expone la lógica de feature analysis, generación de datasets, entrenamiento de modelos, validación de modelos... Posteriormente están los servicios de

web scraping, que se encarga de recolectar todas las noticias nuevas e históricas de nuestra fuente de noticias, y el de sentiment analysis que se encarga de analizar esas noticias.

No solamente tenemos estos 4 servicios si no que nos apoyamos también en 3 complementarios, mysql nuestra base de datos tabular, redis nuestra base de datos key value en memoria y ollama el cual nos permite usar modelos de lenguaje en local fácilmente.

Todos nuestros servicios se comunican entre sí para ofrecer una funcionalidad completa al usuario final.

Ahora le doy paso a mi compañero hua que va a explicar las tecnologías que usamos en cada servicio.

Diapositiva 9 y 10 - Technologies - Hua

For the technologies: on the front-end, React set up with Vite and styled with Tailwind, a very standard stack. The backend is Node.js with Express, well documented and quick to work with. The models service uses NumPy and pandas to process and manage the data, scikit-learn for the machine-learning algorithms, and FastAPI to expose it all as a Python API. The scraper and the sentiment analyser are Python too, so they share the same libraries: the scraper uses BeautifulSoup to pull the headlines out of the HTML, and the analyser talks to Ollama, which is the easiest way to host and run an LLM.

Diapositiva 11 - Webapplication - Eric

Continuando con la Web Application, necesitábamos asociar los modelos y proyectos creados con sus respectivos usuarios. Para ello, era necesario implementar un sistema de autenticación que permitiera identificar a cada usuario dentro de la aplicación.

Una vez registrados y autenticados, los usuarios accedían al dashboard, que actuaba como el punto central desde el que podían gestionar y configurar los diferentes modelos y proyectos que fueran creando.

Para mantener una interfaz organizada y evitar concentrar toda la funcionalidad en una única pantalla, dividimos el dashboard en cuatro secciones. Una estaba dedicada a los proyectos, otra a los modelos, una tercera permitía consultar información rápida de los modelos creados y realizar acciones rápidas sobre ellos, y la última estaba destinada a la información del perfil del usuario.

Ahora que hemos visto cómo el usuario puede gestionar sus proyectos y modelos, el siguiente paso es explicar qué ocurre realmente con esos modelos: Y aquí es donde entra la parte del Machine Learning.

Diapositiva 12 - Financial machine learning - Javier

Para hacer que los modelos de predicción de mercado funcionen correctamente nos hemos tenido que enfrentar a los siguientes obstáculos:

Obtención de datos, para entrenar modelos necesitamos datos históricos de mercado. necesitamos datos de diferentes stocks y que se puedan acceder a ellos de manera consistente. Decidimos usar yahoo finance, nos permite hacer justo esto que estamos buscando e incluye los datos mas recientes del mercado lo que nos permite realizar predicciones con ellos.

Relación señal-ruido baja en los datos. Es el problema más difícil en cuanto a los modelos se refiere. Debido a la naturaleza ruidosa de los datos financieros el mercado es muy difícil de predecir. Para extraer más señal de los datos hemos implementado varios métodos de procesamiento que ofrecemos al usuario como opciones para que construya un modelo a su antojo. Usamos métodos para definir variables objetivo inteligentes y un muestreo por eventos que permite al modelo final rendir mejor. También ofrecemos un módulo de análisis de características para que el usuario final pueda elegir las mejores opciones para su modelo.

Diapositiva 13 - Wizard - Hua

To have model creation no-code, the user still has to go through the whole process: choosing what to predict, selecting features, picking an algorithm and tuning its hyperparameters. The wizard breaks that into four steps.

The first two are the easy ones: name and description, then the stock and how much history the model is trained on.

The third step is the important one. As Javier explained, the target and the event sampling method decide what the model predicts and which days count as samples; in the wizard that pair is a single choice at the top of the step. Right below it the user runs the feature analysis: several algorithms that say which of our thirty-three features, sentiment included, actually carry useful information for that stock and that configuration.

The last step is the algorithm and its hyperparameters. To make that easier we offer automatic optimisation: a grid search with cross-validation that keeps the best combination.

Diapositiva 14 - Webscraping - Eric

Pasando al web scraping, una de las decisiones principales fue desarrollar un scraper genérico. Por lo que, en lugar de crear una implementación diferente para cada fuente, separamos la lógica del scraper de la configuración de cada página

En nuestro caso implementamos TechCrunch como fuente. Utilizamos HTTP requests para obtener las páginas y BeautifulSoup para analizar el HTML y extraer el título y la fecha de las noticias.

También desarrollamos un sistema de scraping histórico mediante paginación y guardamos en la base de datos la última página procesada. Esto permite que, si el proceso se detiene, pueda continuar desde donde se quedó.

Desarrollamos un monitor que revisa periódicamente la primera página para detectar nuevas noticias y que se ejecuta en paralelo con el scraper histórico.

Diapositiva 15 - Sentiment analysis - Hua

After gathering enough headlines with our web scraping module, we need to turn them into data our models can use. We considered NLTK, which scores each word on its own and adds them up, but it has no concept of financial context, so it wasn't usable for us.

Instead we use a large language model. Each headline goes through a Redis queue to the analyser, which asks the model for two scores and forces the answer into a JSON schema, so we always get numbers. Sentiment is the tone of the headline; market impact is how much the model thinks it moves the tech market as a whole.

The hard part was the prompt: it overrated impact, so we wrote a calibrated rubric, with every band anchored to a type of news and explicit rules, like a product update scoring zero. We also make the model reason before it scores, which makes the results much more consistent.

Diapositiva 16 - Report Generation - Eric

En cuanto a la generación de reportes, una de las principales decisiones fue utilizar Ollama para transformar la información obtenida de los modelos en un análisis más fácil de comprender para el usuario.

El proceso comienza cuando el usuario solicita un reporte para un proyecto. Entonces, el backend obtiene los modelos asociados a ese proyecto y recopila la información relevante de cada uno, como el tipo de modelo, el objetivo, las métricas de rendimiento y las predicciones realizadas.

Toda esta información se agrupa y se proporciona al modelo de lenguaje mediante un prompt diseñado específicamente para este propósito. De esta forma, el modelo dispone del contexto necesario para analizar conjuntamente los resultados de los diferentes modelos en lugar de analizar cada resultado de forma aislada.

Finalmente, el servidor de Machine Learning devuelve el análisis generado al backend, que se encarga de integrarlo con la aplicación y mostrar el reporte al usuario.

Docker deployment - Javier

En mitad del desarrollo nos surgió un problema. Para utilizar la aplicación por completo necesitábamos empezar varios procesos manualmente. Para solucionar este problema, estandarizar la ejecución del proyecto y facilitar su inicio creamos imágenes de docker para cada servicio del proyecto y controlamos la interacción entre ellas y su inicialización con docker compose.

Damos pie a la demostración de nuestra aplicación

Diapositiva 17 - Demo

Diapositiva 18 - Conclusions - Javier

Respecto a lo que hemos conseguido

Diapositiva 20 - Future work - Eric

Sobre el trabajo futuro. ¿Cuáles son nuestros siguientes pasos?, pues hay varias mejoras que podrían incorporarse.

En primer lugar, se podría dar al usuario mayor control sobre la temporalidad de los datos, permitiendo seleccionar diferentes timeframes. Esto permitiría utilizar intervalos más pequeños y disponer de un mayor número de muestras para entrenar y analizar los modelos, reduciendo una de las limitaciones actuales.

Otra incorporación importante sería un módulo de backtesting, que permitiría simular como habría funcionado un modelo si realmente hubiéramos utilizado sus predicciones para invertir y, por tanto, conocer su capacidad potencial para generar beneficios.

En cuanto al análisis de noticias, el scraping podría ampliarse para trabajar con múltiples fuentes en lugar de utilizar únicamente TechCrunch, proporcionando más información para el análisis de sentimiento.

Por último, se podría aumentar la personalización de la aplicación, dando al usuario mayor libertad a la hora de crear sus propias estrategias

Diapositiva 21 - Personal Contributions - Todos

“My work mainly covered the web scraping pipeline, the generation of ML reports, and the backend connections required to integrate these functionalities into the application.”